

Aprendizaje Reforzado: Guía 3: Programación dinámica

José Saint Germain

7 de octubre de 2025

1. Ejercicio 1

[Programación] Para el proceso de Markov de recompensas (MRP) de la figura, calcular los valores de los estados de forma iterativa con los siguientes algoritmos y compare sus convergencias. Considere factor de descuento $\gamma < 0,9$.

1. Actualizar todos los valores a la vez por iteración: $\nu_{k+1} = r + \gamma P \nu_k$, con ν_k , r siendo los vectores de valores y recompensas, respectivamente; y P la matriz de probabilidades de transición.
2. Actualizar el valor de un estado por vez (in place): $\nu_{k+1}(s') = r(s') + \gamma \nu_k(s)$, con $\nu_k(s)$ y $r(s')$ siendo los valores y recompensas correspondientes a los estados s y s' , respectivamente.

1.1. Resolución

```
1 R_S2_S1 = 2
2 R_S1_S2 = 1
3
4 P = np.array([
5     [0,1], # S1
6     [1,0]  # S2
7 ])
8
9 def value_iteration(P, R, gamma, theta=1e-6, max_iterations=1000):
10     n_states = P.shape[0]
11     V = np.zeros(n_states)
12
13     for i in range(max_iterations):
14         delta = 0
15         V_new = np.copy(V)
16
17         for s in range(n_states):
18             V_new[s] = R[s] + gamma * np.sum(P[s] * V)
```

```

19         delta = max(delta, abs(V_new[s] - V[s]))
20
21     V = V_new
22
23     if delta < theta:
24         break
25
26     return V
27
28 def value_iteration_in_place(P, R, gamma, theta=1e-6, max_iterations
=1000):
29     n_states = P.shape[0]
30     V = np.zeros(n_states)
31
32     for i in range(max_iterations):
33         delta = 0
34
35         for s in range(n_states):
36             v = V[s]
37             V[s] = R[s] + gamma * np.sum(P[s] * V)
38             delta = max(delta, abs(V[s] - v))
39
40         if delta < theta:
41             break
42
43     return V
44
45 R = np.array([R_S1_S2, R_S2_S1]) # Recompensas para S1 y S2
46 gamma = 0.9
47
48 V1 = value_iteration(P, R, gamma)
49 V2 = value_iteration_in_place(P, R, gamma)
50 print("Valores de estados (actualizacion simultanea):", V1)
51 print("Valores de estados (actualizacion in place):", V2)

```

```

1  Valores de estados (actualizacion simultanea): [7.22221972 7.77777546]
2  Valores de estados (actualizacion in place): [7.22222062 7.7777765 ]

```

2. Ejercicio 2

En el ejemplo 4.1 (GridWorld, Sutton & Barto, 2018), donde la política π es aleatoria y equiprobable,

1. ¿Cuánto vale $q_\pi(11, \text{down})$?
2. ¿Cuánto vale $q_\pi(7, \text{down})$?

Nota: Utilize $v(11) = -14$ (fig 4.1 del libro)

Justifique sus respuestas.

2.1. Resolución

Sabemos que $q_\pi(s, a) = r + \gamma \nu_\pi(s')$, donde la recompensa siempre es -1 y no es descontada.

$$q_\pi(s, a) = -1 + \nu_\pi(s')$$

$$q_\pi(11, \text{down}) = -1 + \nu_\pi(15) = -1 + 0 = -1$$

$$q_\pi(7, \text{down}) = -1 + \nu_\pi(11) = -1 - 14 = -15$$

3. Ejercicio 3

En el ejemplo 4.1 (GridWorld, Sutton & Barto, 2018), suponga que se agrega un nuevo estado 15 debajo del estado 13 y sus acciones: left, up, right y down, lleva al agente a los estados 12,13,14,15, respectivamente.

Considere que las transiciones desde los estados originales no se cambian. ¿Cuánto vale $\nu_\pi(15)$ para la política aleatoria y equiprobable? Utilice

- $\nu(12) = -22$
- $\nu(13) = -20$
- $\nu(14) = -14$ (fig 4.1 del libro).

Justifique su respuesta.

3.1. Resolución

Sabemos que:

$$\nu_\pi(s) = \sum_a \pi(a|s) [r(s, a) + \gamma \nu_\pi(s')]$$

si la política es aleatoria y equiprobable, y la recompensa siempre es -1:

$$\nu_\pi(s) = \frac{1}{4} \sum_a [-1 + \nu_\pi(s')]$$

Entonces, $\nu_\pi(15)$ vale:

$$\nu_\pi(15) = \frac{1}{4}[-1 + \nu_\pi(12) + -1 + \nu_\pi(13) + -1 + \nu_\pi(14) + -1 + \nu_\pi(15)]$$

tomamos factor común:

$$\nu_\pi(15) = -1 + \frac{1}{4}[\nu_\pi(12) + \nu_\pi(13) + \nu_\pi(14) + \nu_\pi(15)]$$

reemplazamos con los valores conocidos:

$$\nu_\pi(15) = -1 + \frac{1}{4}[-22 - 20 - 14 + \nu_\pi(15)]$$

$$\nu_\pi(15) = -1 + \frac{1}{4}[-56 + \nu_\pi(15)]$$

$$\nu_\pi(15) - \frac{1}{4}\nu_\pi(15) = -1 - 14$$

$$\frac{3}{4}\nu_\pi(15) = -15$$

$$\nu_\pi(15) = -20$$

4. Ejercicio 4

En el Ejemplo 4.3 (Gambler's problem, Sutton&Barto, 2018) la política óptima tiene una forma particular (ver figura) con máximo en 50. Es decir, cuando el jugador tiene \$50, le conviene apostar todo; sin embargo, cuando tiene \$51, le conviene apostar \$1. ¿Por qué sucede esto?

4.1. Resolución

La moneda no está equilibrada y las probabilidades están en contra del jugador. Por lo tanto, a medida que la cantidad de lanzamientos aumenta, la probabilidad de ganar disminuye. El jugador debe, entonces, jugar con la menor cantidad de lanzamientos posibles: debería apostar todo su dinero cuando tiene 50, ya que con un lanzamiento puede llegar a 100 y ganar el juego. Caso contrario, debe apostar lo mínimo para no perder sus posibilidades de ganar.

5. Ejercicio 5

[Programación] Implemente el Algoritmo de Iteración de Valores para el Ejemplo 4.3 (Gambler's problem, Sutton&Barto, 2018) para los siguientes casos:

1. $p(\text{CARA}) = 0.25$
2. $p(\text{CARA}) = 0.55$

Compare los resultados y saque conclusiones.

5.1. Resolución

Evaluando la primera situación ($p(\text{CARA}) = 0.25$) en la figura 1, vemos que la política óptima no se modifica con respecto al punto anterior ($p(\text{CARA}) = 0.4$). Esto se debe a que la probabilidad de ganar sigue siendo baja y hay que lograr el objetivo con la menor cantidad de lanzamientos posibles.

Para el caso de la figura 2 ($p(\text{CARA}) = 0.55$), la política óptima cambia considerablemente. Ahora, la probabilidad de ganar es mayor y el jugador puede darse el lujo de apostar el mínimo dinero indispensable (1 dólar) para llegar a 100. De ahí que, sin importar el capital acumulado, siempre convenga apostar un sólo dólar.

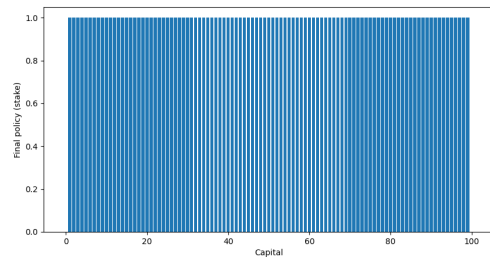
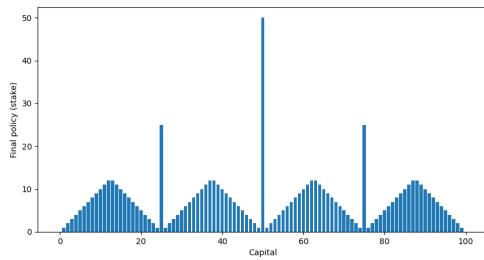


Figura 1: Política óptima $p(\text{CARA}) = 0.25$

Figura 2: Política óptima $p(\text{CARA}) = 0.55$